

1. Toom-3 (Toom–Cook 3-way) integer multiplication

Sources: A. Toom (1963); S. Cook, PhD thesis (1966); interpolation sequence from M. Bodrato, WAIFI 2007 (as given on the Wikipedia article). Code: `toom3/toom3.py`. Theory: $T(n) = 5T(n/3) + O(n) \Rightarrow \Theta(n^{\log_3 5}) \approx \Theta(n^{1.465})$.

Each n -bit operand is cut into three $\lceil n/3 \rceil$ -bit pieces and treated as a degree-2 polynomial in $B = 2^k$. Both polynomials are evaluated at 0, 1, -1 , -2 and ∞ , the five point values are multiplied recursively (instead of the 9 products a schoolbook split needs), the degree-4 product is recovered by exact interpolation, and the result is reassembled with shifts and adds. Operands of 64 bits or less are multiplied directly.

What I learned

- Toom-3 is the same idea as Karatsuba pushed one step further: fewer recursive products in exchange for more linear-time additions. That trade only pays off asymptotically, which is why real libraries switch between methods by size.
- The evaluation points -1 and -2 make the sub-operands negative and a few bits *longer* than a piece ($|p(-2)| < 7 \cdot 2^k$). My first version recursed forever with a tiny cutoff, because a 5-bit number does not get shorter when you split it. I added a minimum cutoff and handle signs explicitly.
- When benchmarking in Python you have to make sure you are timing your recursion and not the interpreter's. CPython already switches to its own Karatsuba above 70 30-bit digits (2100 bits). Keeping my base case at 64 bits means the leaves are tiny schoolbook multiplies and the exponent you see is Toom-3's. Python also refuses to convert ints of more than 4300 decimal digits to strings, so the test files store numbers in hex.
- The $\div 3$ in the interpolation is exact, and CPython divides by a one-digit divisor in linear time, so it does not break the $O(n)$ combine step.

How I created the test cases

`toom3/generate_tests.py` (seeded, so it is reproducible) writes 10 cases, each holding two random operands of exactly $n = 3^j$ bits for $j = 4, \dots, 13$ (81 to 1,594,323 bits, top bit forced to 1), plus their product. I used powers of three on purpose. With a size cutoff, the runtime of a divide-and-conquer algorithm is a staircase: the number of leaves jumps by $5 \times$ each time n crosses cutoff $\cdot 3^L$. Powers-of-3 sizes hit every stair at the same relative spot, so each case is exactly one recursion level deeper than the last. The expected product comes from Python's built-in multiplication, which shares no code with mine. The runner checks every product and reports the minimum time over at least 3 runs, repeating small cases for at least 1 s, with the garbage collector off. All timings were taken on an Apple M1 Max with Python 3.12. Separate unit tests (`test_toom3.py`) cover signs, zero, all-ones operands, very unbalanced sizes, several cutoffs, and 2000 random pairs.

Runtime vs. theory

Figure 1 (next page) shows the measured times on log–log axes with $c \cdot n^{\log_3 5}$, where c is fit on the five largest cases. A least-squares fit to those five points has slope **1.469**, against the theoretical 1.465. It is clearly separated from Karatsuba (1.585) and schoolbook (2), whose lines are drawn through the largest point for comparison. Put another way, every $3 \times$ increase in n multiplied the time by 4.97–5.05 from case 4 on, and $5 = 3^{\log_3 5}$ is exactly the theoretical ratio. The normalized plot $T/(c \cdot n^{1.465})$ stays within 3% of 1 from 729 bits up to 1.6 million bits.

Only the two smallest cases sit below the line (0.79 and 0.91), and the recurrence predicts that. Solving $T(n) = 5T(n/3) + \beta n$ exactly gives $T(n) = C \cdot n^{\log_3 5} - 1.5\beta n$. The linear term (the additions and shifts) is a lower-order correction. It pulls the smallest inputs slightly below the leading term and vanishes relative to it as n grows, which is what the bottom panel shows. Because the fitted exponent, the per-step ratio and the flat normalized curve all agree, I believe this implementation does match the theoretical $\Theta(n^{\log_3 5})$ bound.

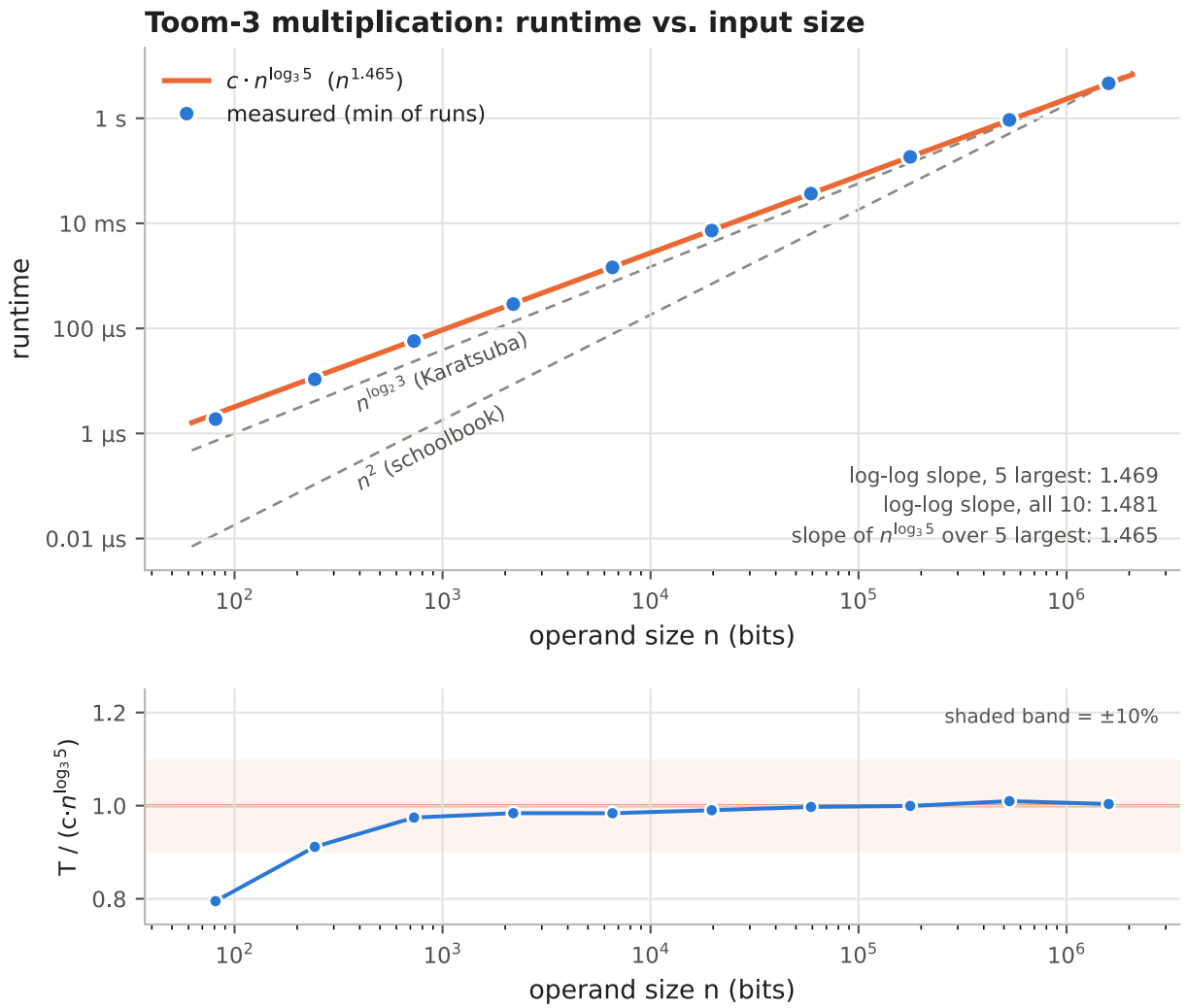


Figure 1. Toom-3: runtime vs. operand size (top, log-log) with the fitted theoretical curve $c \cdot n^{\log_3 5}$ (orange) and other growth rates through the largest point (gray, dashed). Bottom: runtime divided by the fitted curve. A flat line at 1 means the implementation grows exactly like $n^{1.465}$.

2. Hirschberg's linear-space longest common subsequence

Source: D. S. Hirschberg, "A linear space algorithm for computing maximal common subsequences," CACM 18(6), 1975. The paper's Algol-style ALG A/B/C are translated to Python. Code: `hirschberg/hirschberg.py`. Theory: $\Theta(mn)$ time, $\Theta(m+n)$ space.

ALG B computes the last row of the LCS-length table keeping only two rows. ALG C splits A at $i = \lfloor m/2 \rfloor$ and runs ALG B forward on $A[:i]$ and backward on the reversed $A[i:]$ and B. It then takes the first column k that maximizes $L_1[k] + L_2[n-k]$, which is where an optimal path crosses row i , and recurses on $(A[:i], B[:k])$ and $(A[i:], B[k:])$. The two subproblems have total area $i \cdot k + (m-i)(n-k) \approx mn/2$ *wherever* k falls, so the work is $mn + mn/2 + mn/4 + \dots \leq 2mn$.

What I learned

- The main point is that the cut column k can be found with two linear-space passes (forward and reversed). And because the subproblem areas always halve, recomputing values instead of storing them costs at most a constant $2\times$ in time while dropping memory from $\Theta(mn)$ to $\Theta(m+n)$. I measured it: on the 4096×4120 case the full table peaked at 254 MB and Hirschberg at 0.38 MB, $667\times$ less (Figure 2, bottom).
- The indexing is easy to get wrong. The backward pass is over reversed suffixes, so the score for column j is $L_2[n-j]$, not $L_2[j]$. I wrote unit tests that compare ALG B's row against the full table cell by cell.
- An interpreter detail can show up in a runtime plot. CPython preallocates the integers $-5\dots 256$, so every DP value above 256 has to allocate a new int object. I measured 40.9 ns per cell with small values and 49.0 ns per cell with every value forced above 256, a 20% difference that comes from the values alone and not the algorithm.
- The order of measurements matters. At first I measured memory in between the timings, and the cases timed right after a 250 MB table had been allocated and freed ran about 10% slower. That alone pushed the slope from 1.016 to 1.034. Now the runner does all the timing first and measures memory in a separate pass.

How I created the test cases

`hirschberg/generate_tests.py` (seeded) writes 10 pairs of DNA strings whose product mn doubles each case, from 512×512 (2.6×10^5 cells) to 16384×8192 (1.3×10^8 cells). I mixed the shapes (A longer, B longer, square) to check that only the product mn matters. I also alternated two kinds of input: independent random strings, and $B = A$ with 10% random substitutions, insertions and deletions, which looks like aligning two similar sequences. The split column k depends on the content, so this checks that the runtime does not. The expected LCS length comes from a bit-parallel LCS algorithm (Allison-Dix / Hyrö) that uses Python big ints as bit vectors. It shares no code with my implementation, and the generator also checks it against the plain DP for cases up to 2.5M cells. Because an LCS is not unique, an answer passes if it is a subsequence of both strings and has the expected length. Timing works as in Section 1. Unit tests cover empty and single-character strings, the CLRS examples, identical and disjoint strings, and 1500 random pairs checked against ALG A.

Runtime vs. theory

In Figure 2 the points follow $c \cdot mn$ across 2.7 orders of magnitude. The log-log slope is **1.016** over the five largest cases and 1.012 over all ten, and each doubling of mn multiplied the time by 1.90–2.13. The normalized ratio $T/(c \cdot mn)$ stays between 0.91 and 1.05, with no pattern by shape or by random-vs-related input. An extra log factor would be visible: $mn \log(mn)$ has a local slope of about 1.055 over the large cases, and the measured slope is much closer to 1. What remains is a slight upward creep at the large end (time per mn grows about 16% from case 4 to case 10), and the small-int effect above explains it. In the smallest cases the DP values only reach 334–963, so a large share of cells hold cached ints. In the largest cases values reach 7746 and nearly every cell allocates one. That per-cell cost is capped at about +20%, so it cannot add a growing factor, and the algorithm has no log factor anywhere (the area argument above). The memory panel confirms the other half of the claim: Hirschberg's peak grows with $m+n$ (0.02→0.38 MB) while the full table grows with mn (2.3→254 MB). I believe the implementation matches $\Theta(mn)$ time and $\Theta(m+n)$ space.

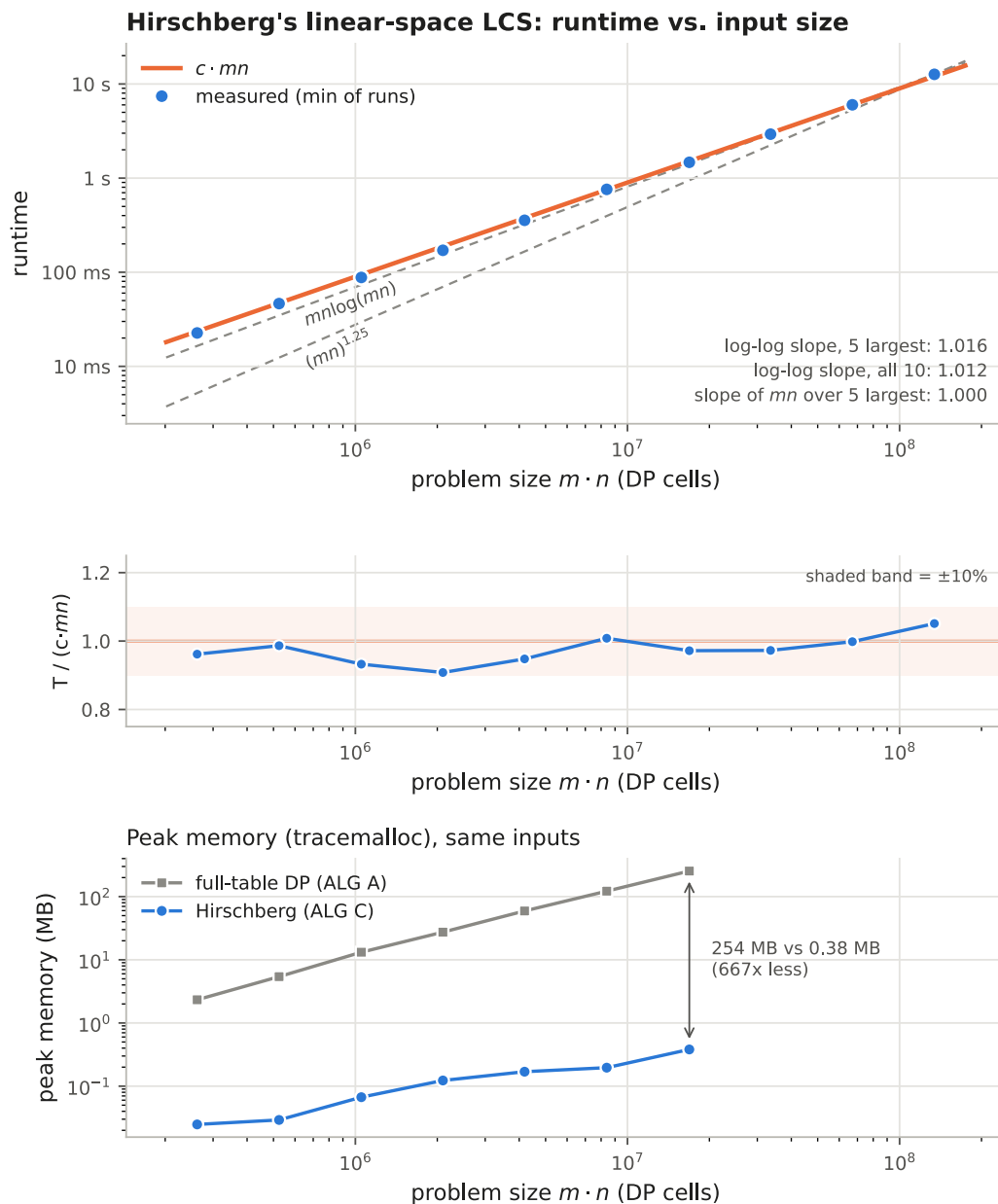


Figure 2. Hirschberg: runtime vs. $m \cdot n$ with the fitted $c \cdot mn$ line (top), runtime divided by the fit (middle), and peak memory measured with `tracemalloc` for Hirschberg vs. the full-table DP on the first seven cases (bottom).

3. Closest pair of points (divide and conquer)

Sources: M. Shamos & D. Hoey, “Closest-point problems,” FOCS 1975; Preparata & Shamos, *Computational Geometry* (1985) §5.4; CLRS §33.4. Code: `closest_pair/closest_pair.py`. Theory: $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$.

The algorithm sorts the points by x once, splits at the median, and solves both halves to get $d = \min(d_L, d_R)$. The only remaining candidates lie in the strip $|x - x_{\text{mid}}| < d$. Walking the strip in y order, each point only needs to be compared with the next few points (at most 7, by a packing argument), and the loop stops as soon as $\Delta y^2 \geq d^2$. Each call returns its points merged by y , so the strip never has to be re-sorted.

What I learned

- Sorting the strip by y at every level gives $O(n \log^2 n)$, not $O(n \log n)$. The fix is to run merge sort on y alongside the recursion: each call returns its points sorted by y and the parent merges the two lists in $O(n)$. That is what makes the combine step linear and the recurrence $2T(n/2) + O(n)$.
- Ties in x are a classic bug. I split by index, not by x value, and build the strip from the merged list, so points sharing $x = x_{\text{mid}}$ can end up on either side and still get compared. A unit test puts every point on a single vertical line (the whole input is in the strip at every level) and checks against brute force. I also used squared integer distances so every comparison is exact.
- The RAM model assumes every memory access costs the same, and for Python objects that clearly stops being true once the data outgrows the cache (below).

How I created the test cases

`closest_pair/generate_tests.py` (seeded) writes 10 cases with $n = 2^{10}, \dots, 2^{19}$ points (1,024 to 524,288) with uniform random integer coordinates in $[0, 2^{24})^2$. All ten use the same distribution so that only n changes between cases. I tried clustered and narrow-band inputs at $n = 2^{17}$ and they changed the time by less than 10%. The adversarial layouts (one line, duplicates, grids, a pair straddling the split, floats, clusters) are in `test_closest_pair.py` and are checked against brute force. The expected answer comes from scipy's k-d tree: I take each point's nearest neighbour and the minimum is recomputed exactly in integers. That is completely independent of my code. A case passes if the squared distance matches and the two returned points are input points at that distance. The initial sort is timed and file parsing is not. Otherwise timing works as in Section 1.

Runtime vs. theory

Figure 3 shows the measured times against $c \cdot n \log_2 n$. From 1,024 to 65,536 points, $T/(n \log_2 n)$ is constant within $\pm 3\%$ (about 205–216 ns per $n \log_2 n$), and each doubling cost 2.13–2.18 $\times$, where $n \log n$ predicts 2.13–2.20. In the last three cases the ratio climbs to 1.15 and the local slope is 1.17, against 1.085 for $n \log n$ over that range. My explanation is the memory hierarchy, not a different asymptotic class. Each point is a tuple plus two int objects (about 112 bytes, plus list pointers), so the working set outgrows the M1 Max's 12 MB L2 cache at roughly 100k points, which is exactly where the rise starts. After that, every recursion level whose subproblem no longer fits in cache pays misses on every point, and the number of such levels, $\log(n/M)$, keeps growing: an extra $(n/B) \cdot \log(n/M)$ term in the external-memory model. Two checks support this. Python's own `list.sort`, also $n \log n$, drifted +12% per $n \log n$ from 1k to 1M points on the same kind of data. And allocating the points in x -sorted memory order (better locality) cut my 1M-point time by 10%. An $O(n^2)$ algorithm would grow 4 $\times$ per doubling, and an $O(n \log^2 n)$ one would need 2.3–2.4 $\times$ at the small sizes, where I measured 2.13–2.18 $\times$. Here the extra growth only appears after the cache fills. So I believe the implementation matches $\Theta(n \log n)$ in the RAM model, and the remaining deviation comes from the machine.

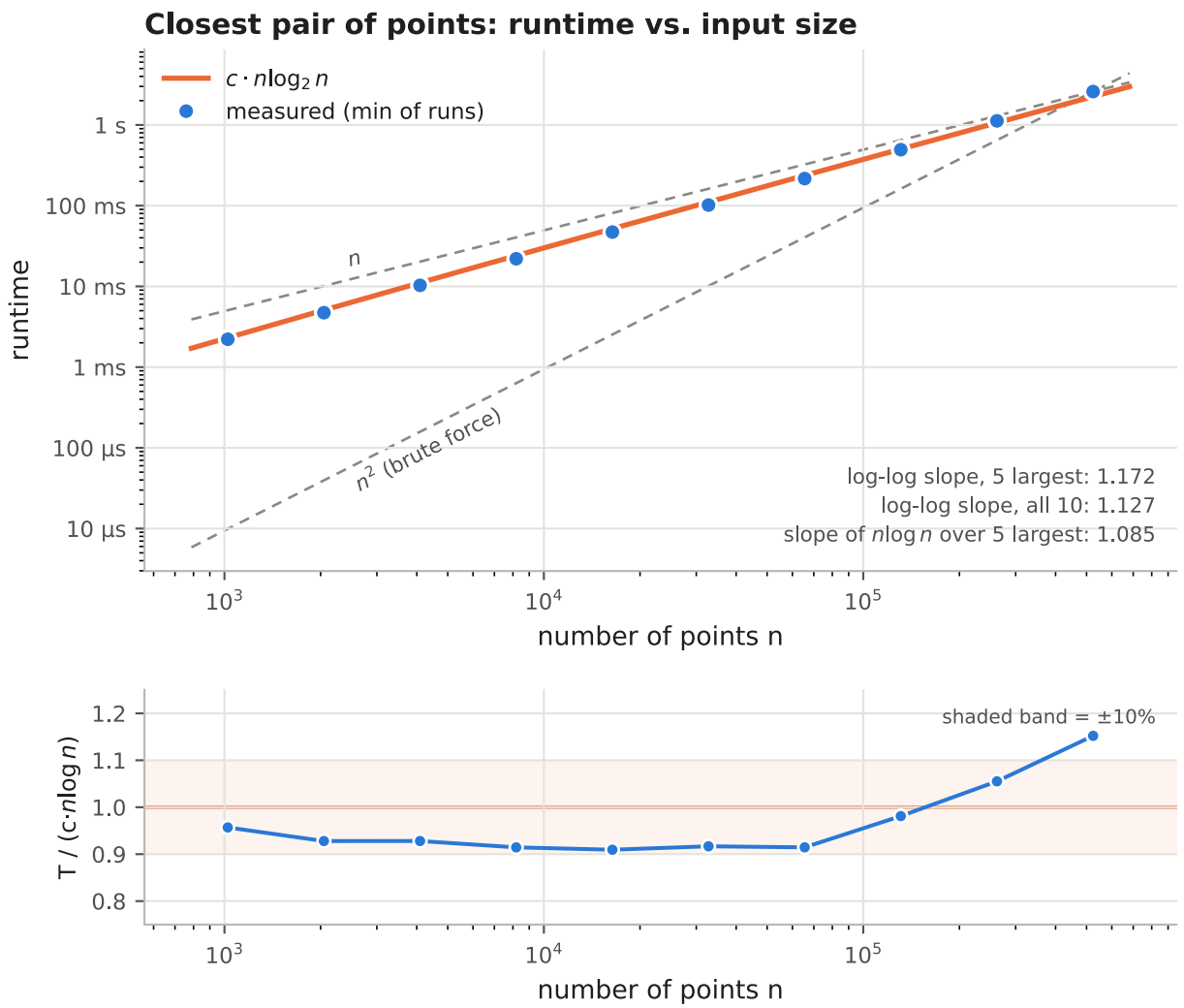


Figure 3. Closest pair: runtime vs. n with the fitted $c \cdot n \log_2 n$ curve (top), and runtime divided by the fit (bottom). The ratio is flat up to 65,536 points; the rise after that starts where the working set exceeds the L2 cache.